

SellFlow

Plataforma E-Commerce

Fecha	7/5/2026
Autor	Jorge Vanegas Joseph Rosas
Plataformas	Android + Web (Laravel API)

1. Descripción General del Proyecto

SellFlow es una plataforma de comercio electrónico compuesta por una aplicación Android y una API RESTful construida en Laravel. El sistema permite a los usuarios navegar catálogos de productos, gestionar carritos de compra, realizar pagos con tarjeta, y a los administradores gestionar el inventario y los pedidos desde un panel de control dedicado.

Repositorio del proyecto

Nombre: proyecto1-android

Stack principal: Kotlin (Android) + PHP Laravel (Backend)

Entorno local: XAMPP (Apache + MySQL)

Repositorio: <https://github.com/josephr04/ds9p1.git>

1.1 Alcance

El sistema cubre los siguientes módulos funcionales:

- Autenticación y gestión de sesión (registro, login, logout)
- Catálogo de productos con búsqueda y filtrado por categoría
- Carrito de compras con actualización de cantidades
- Checkout con formulario de dirección y validación de tarjeta de crédito
- Panel de administración con gestión de productos, categorías y pedidos
- Historial de pedidos por usuario

2. Arquitectura del Sistema

SellFlow adopta una arquitectura cliente-servidor. La aplicación Android actúa como cliente que consume la API REST de Laravel. La comunicación se realiza mediante HTTP/JSON con autenticación basada en tokens Bearer (Laravel Sanctum).

CAPA PRESENTACIÓN	CAPA LÓGICA / API	CAPA DE DATOS
Android App (Kotlin) Fragments + ViewModels Jetpack Navigation Retrofit (HTTP) Room (SQLite local)	Laravel 10 API Controllers / Routes Middleware (auth:sanctum) Policies / Roles Eloquent ORM	MySQL (XAMPP) Migraciones Laravel Seeders / Factories Room DB (SQLite local)

2.1 Stack Tecnológico

Componente	Tecnología	Descripción
App móvil	Kotlin + Android SDK	App nativa Android con MVVM
UI Android	XML Layouts + Material Design 3	Fragments, RecyclerView, ViewBinding
Navegación	Jetpack Navigation Component	NavGraph con bottom navigation
HTTP Client	Retrofit 2 + Gson Converter	Llamadas a la API REST
Persistencia local	Room (SQLite)	Entidades: User, CartItem, etc.
Backend API	Laravel 10 (PHP 8.x)	REST API con Sanctum auth
Base de datos	MySQL 8.x vía XAMPP	Gestionada con Eloquent ORM
Autenticación	Laravel Sanctum	Tokens Bearer por usuario/rol
Entorno local	XAMPP + Apache	Servidor de desarrollo local

3. Arquitectura Android

3.1 Estructura de Archivos

Organización del proyecto Kotlin

com.example.proyecto1android/

```

├── data/
│   ├── api/      → Interfaces Retrofit (ApiService)
│   ├── model/    → Data classes (Product, User, Order, CartItem...)
│   ├── repository/ → Repositories (ProductRepository, OrderRepository...)
│   └── local/    → Room Database, DAOs, Entities
├── ui/
│   ├── auth/     → LoginFragment, RegisterFragment
│   ├── home/     → HomeFragment, ProductListFragment
│   ├── product/  → ProductDetailFragment
│   ├── cart/     → CartFragment, CartViewModel
│   ├── checkout/ → CheckoutFragment, PaymentFragment
│   ├── orders/   → OrdersFragment, OrderDetailFragment
│   └── admin/    → AdminDashboard, ProductMgmtFragment, OrderMgmtFragment
├── viewmodel/    → ViewModels compartidos
└── util/         → Constants, Extensions, Validators

```

4. API REST — Laravel Backend

El backend expone una API RESTful en JSON. Todos los endpoints protegidos requieren el encabezado Authorization: Bearer {token} obtenido al autenticarse. El servidor corre localmente en XAMPP en `http://10.0.2.2:8000/api` para el emulador Android.

4.1 Autenticación

Método	Endpoint	Descripción	Auth	Respuesta
POST	/api/register	Registrar nuevo usuario	No	201 + token
POST	/api/login	Iniciar sesión	No	200 + token + rol
POST	/api/logout	Cerrar sesión	Bearer	200 message
GET	/api/user	Datos del usuario actual	Bearer	200 user object

4.2 Productos

Método	Endpoint	Descripción	Auth	Rol
GET	/api/products	Listar todos los productos	No	Público
GET	/api/products/{id}	Detalle de un producto	No	Público
GET	/api/products?category={id}	Filtrar por categoría	No	Público
POST	/api/admin/products	Crear producto	Bearer	Admin
PUT	/api/admin/products/{id}	Actualizar producto	Bearer	Admin
DELETE	/api/admin/products/{id}	Eliminar producto	Bearer	Admin

4.3 Carrito de Compras

Método	Endpoint	Descripción	Auth	Rol
GET	/api/cart	Ver carrito del usuario	Bearer	Cliente
POST	/api/cart	Agregar ítem al carrito	Bearer	Cliente
PUT	/api/cart/{id}	Actualizar cantidad	Bearer	Cliente
DELETE	/api/cart/{id}	Eliminar ítem del carrito	Bearer	Cliente
DELETE	/api/cart	Vaciar carrito completo	Bearer	Cliente

4.4 Pedidos (Orders)

Método	Endpoint	Descripción	Auth	Rol
POST	/api/orders	Crear pedido desde carrito	Bearer	Cliente
GET	/api/orders	Historial de pedidos del usuario	Bearer	Cliente
GET	/api/orders/{id}	Detalle de pedido	Bearer	Cliente
GET	/api/admin/orders	Todos los pedidos (admin)	Bearer	Admin
PUT	/api/admin/orders/{id}	Actualizar estado del pedido	Bearer	Admin

4.5 Categorías

Método	Endpoint	Descripción	Auth	Rol
GET	/api/categories	Listar categorías	No	Público
POST	/api/admin/categories	Crear categoría	Bearer	Admin
PUT	/api/admin/categories/{id}	Editar categoría	Bearer	Admin
DELETE	/api/admin/categories/{id}	Eliminar categoría	Bearer	Admin

5. Módulos Funcionales

5.1 Autenticación y Roles

El sistema implementa autenticación con Laravel Sanctum. Al hacer login exitoso, el servidor retorna un token personal de acceso que la app Android almacena localmente (SharedPreferences o Room). Las rutas en api.php están agrupadas con middleware auth:sanctum para proteger los endpoints privados.

Roles del sistema

cliente — Puede navegar el catálogo, gestionar su carrito y realizar pedidos.

admin — Accede al panel de administración. Puede crear/editar/eliminar productos, categorías y cambiar el estado de los pedidos.

La distinción de rol se hace en el middleware del backend y en la lógica de navegación del NavGraph Android.

5.2 Catálogo de Productos

El HomeFragment muestra la lista de productos activos usando un RecyclerView con CardViews. Implementa filtrado por categoría (horizontal scrollable chips) y búsqueda por nombre. Los datos se obtienen de la API a través de ProductRepository usando Retrofit y se exponen al Fragment mediante LiveData desde el ProductViewModel.

5.3 Carrito de Compras

El CartFragment muestra los ítems actuales del carrito con opciones para modificar cantidad o eliminar ítems. El CartViewModel mantiene el estado del carrito en memoria y sincroniza con la API. Al presionar 'Proceder al pago', se navega al CheckoutFragment pasando el resumen del carrito como argumento del NavGraph.

5.4 Flujo de Checkout y Pago

El CheckoutFragment presenta un formulario con campos de dirección de envío. Al confirmar la dirección, navega al PaymentFragment donde el usuario ingresa los datos de tarjeta de crédito. La validación de tarjeta incluye:

- Número de tarjeta: Algoritmo de Luhn para validar el número de 16 dígitos
- Fecha de vencimiento: Verificación de que la fecha no esté vencida (MM/YY)
- CVV: Longitud entre 3 y 4 dígitos
- Nombre en tarjeta: Campo obligatorio no vacío

Al confirmar el pago exitoso, el CheckoutViewModel llama al endpoint POST /api/orders, recibe el ID del nuevo pedido y navega a una pantalla de confirmación.

5.5 Panel de Administración

Los usuarios con rol admin son redirigidos al AdminDashboardFragment. El panel incluye:

- Resumen estadístico: total de pedidos, pedidos pendientes, productos en stock
- Gestión de productos: CRUD completo con formulario de alta/edición
- Gestión de pedidos: Vista de todos los pedidos con opción para cambiar estado
- Gestión de categorías: Creación y edición de categorías

6. Configuración del Entorno de Desarrollo

6.1 Backend Laravel (XAMPP)

1. Instalar XAMPP e iniciar Apache y MySQL
2. Clonar el repositorio del backend en la carpeta htdocs
3. Copiar .env.example a .env y configurar la conexión a la base de datos

4. Dirigirse al directorio: C:\xampp\htdocs\ds9p1\backend
5. Ejecutar: composer install
6. Ejecutar: php artisan serve

Variables de entorno críticas (.env)

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=sellflow_db
DB_USERNAME=root
DB_PASSWORD=
SANCTUM_STATEFUL_DOMAINS=localhost
SESSION_DRIVER=cookie
```

6.2 App Android

7. Abrir el proyecto proyecto1-android en Android Studio
8. Verificar que el emulador Android esté configurado
9. Configurar la BASE_URL en Constants.kt: http://10.0.2.2:8000/api/
10. Ejecutar Gradle Sync para descargar dependencias
11. Ejecutar la app en el emulador (asegurarse de que el backend esté corriendo)

Dependencias principales en build.gradle (app)

```
implementation 'com.squareup.retrofit2:retrofit:2.9.0'
implementation 'com.squareup.retrofit2:converter-gson:2.9.0'
implementation 'androidx.room:room-runtime:2.6.1'
kapt 'androidx.room:room-compiler:2.6.1'
implementation 'androidx.navigation:navigation-fragment-ktx:2.7.x'
implementation 'androidx.lifecycle:lifecycle-viewmodel-ktx:2.7.x'
implementation 'com.github.bumptech.glide:glide:4.16.0'
```